

StockFlow — Dossier projet Bloc

4

RNCP 39583 niveau 7 — Maintenir l'application logicielle en condition opérationnelle

Édouard Sence
Août 2026

Sommaire

1. **Introduction** — le projet, le périmètre de ce dossier
2. **Processus de mise à jour des dépendances**
3. **Système de supervision et de signalement (C4.1.2)**
4. **Collecte et consignation des anomalies (C4.2.1)**
5. **Fiche de consignation d'une anomalie réelle**
6. **Traitement d'une anomalie détectée**
7. **Recommandations d'amélioration**
8. **Journal des versions (C4.3.2)**
9. **Problème résolu avec le support client**
10. **Conclusion**
11. **Annexe** — correspondance pièces officielles ↔ dossier technique

Introduction

StockFlow est une application web de gestion de parc informatique pour TPE/PME : inventaire des équipements, étiquetage QR, scan mobile, signalement et suivi des incidents — y compris hors connexion (PWA). L'application est en production sur Vercel (base PostgreSQL Supabase), version stable `v0.4.0` au moment de la rédaction — code source : <https://github.com/EdouardSence/StockFlow>, production : <https://stock-flow-gamma-eight.vercel.app>. La conception, la sécurisation et la recette sont traitées dans le dossier Bloc 2 ; ce dossier ne les répète pas — il documente **comment l'application est maintenue en condition opérationnelle** : mise à jour, supervision, détection et traitement des anomalies, gestion des versions, recours au support éditeur.

La discipline est la même que dans le dossier Bloc 2 : tout ce qui est présenté ici est **réel et vérifiable** (configurations lues sur les instances, incidents datés avec leurs commits, tickets avec leurs URL). Ce qui n'est pas fait est écrit comme tel.

1. Processus de mise à jour des dépendances

Le référentiel des composants tiers et sa politique de suivi sont tenus dans le dépôt (`docs/certification/07-referentiel-composants.md`). Le processus opérationnel :

Fréquence, périmètre, type : les mises à jour sont **manuelles** (pas de bot type Dependabot/Renovate — choix assumé pour un projet mono-développeur : chaque montée est relue, pas fusionnée en masse), à **cadence mensuelle**, avec montée **immédiate** en cas d'alerte de sécurité touchant le runtime. Le périmètre couvert est l'ensemble des dépendances déclarées dans `package.json` (runtime et outillage), résolues et verrouillées par `bun.lock`.

1. **Versions verrouillées** : `bun.lock` est commité — chaque environnement (dev, CI, production Vercel) installe exactement les mêmes versions résolues. Aucune montée de version implicite.
2. **Montée de version = commit dédié**, qui doit passer les mêmes garde-fous que tout changement : hooks locaux (lint + typecheck, message de commit), CI GitHub Actions (lint → typecheck → tests → build), et les suites locales si le runtime est touché (99 tests Vitest dont l'intégration RLS ; 36 scénarios e2e Playwright, exécutés en local uniquement car la base est partagée — dette documentée au Bloc 2).
3. **Audit de vulnérabilités à chaque montée** : `bun audit`. Premier inventaire le 2026-07-07 : 16 vulnérabilités transitives, tracées (issue #24) et différées par choix argumenté. Ré-audit du 2026-08-13 pour ce dossier : **40 vulnérabilités** (1 critique, 24 hautes, 12 modérées, 3 faibles) — chaque chemin de dépendance re-vérifié : toutes restent confinées à l'outillage de build et de dev (Vite, devtools, ESLint/commitlint, jsdom, workbox-build), **aucune n'atteint le runtime de production**. L'issue #24 est commentée à chaque re-mesure : la décision de différer est datée et révisable, pas oubliée.
4. **Pas de nouvelle dépendance sans justification** : la fonctionnalité offline (file d'incidents) a été livrée avec zéro dépendance ajoutée (IndexedDB natif plutôt qu'une librairie de file d'attente).
5. **Points d'attention spécifiques, appris d'incidents réels** : après toute montée touchant Vite/Nitro/ `vite-plugin-pwa`, vérifier que le build copie toujours `sw.js` dans la sortie statique — la cible dépend du preset Nitro (`.vercel/output/static/` sur Vercel, `.output/public/` en local), et l'issue #34 a montré qu'une cible codée en dur casse silencieusement tous les déploiements alors que la CI GitHub reste verte. Vérifier le statut Vercel après push fait partie du processus.

6. **Ordre en cas de migration de schéma** : la migration (rétro-compatible) est appliquée d'abord, le code qui l'exploite est déployé ensuite. Jamais d'édition d'une migration déjà appliquée — une erreur se corrige en avant.

2. Système de supervision et de signalement (C4.1.2)

Pièce détaillée : `docs/certification/22-supervision-alerte.md` — état vérifié sur l'instance Sentry réelle le 2026-07-15, par l'API, pas une description d'intention.

Périmètre et indicateurs

Les erreurs runtime côté client (exceptions non attrapées, erreurs d'hydratation React) sont capturées par `@sentry/react` (SaaS, région UE), initialisé côté client uniquement avec `sendDefaultPii: false` (minimisation RGPD, correctif issu de la revue de sécurité du Bloc 2). Indicateurs suivis : nouvelles erreurs (groupées par empreinte), fréquence et escalade d'une erreur existante, utilisateurs touchés, régressions. Ce périmètre est proportionné à un MVP mono-développeur : le client est là où vivent les parcours critiques (scan, saisie offline) et où les erreurs sont invisibles autrement ; les logs de fonctions Vercel couvrent le reste.

Modalité de signalement configurée

Règle d'alerte **active** sur le projet Sentry (ID 541468, état `enabled`) :

Élément	Valeur vérifiée
Déclencheurs	Nouvelle issue de haute priorité ou issue existante repassant en haute priorité (la détection d'escalade couvre les pics de fréquence anormaux)
Action	Email immédiat aux membres actifs (délai de regroupement : 0 min)

Le couple « nouvelle erreur » + « escalade » couvre les deux modalités attendues — signalement de l'inédit et signalement du pic — avec le modèle adaptatif de Sentry plutôt qu'un seuil arbitraire.

Surveillance de la disponibilité

Sentry surveille les erreurs ; la **disponibilité** est surveillée par une sonde dédiée (`.github/workflows/uptime.yml`) : un workflow planifié vérifie **toutes les 15 minutes** que l'URL de production répond HTTP 200 (3 tentatives espacées, pour ne pas alerter sur un raté réseau transitoire). Un échec déclenche la notification email GitHub et fait passer au rouge le badge « Uptime » du README — le même principe que le badge CI né de l'incident #26 (§ 5) : un rouge doit se voir. La sonde détecte ce qu'aucune erreur cliente ne peut signaler : une panne totale où plus rien ne se charge.

Preuve de fonctionnement bout en bout

- Dernier déclenchement réel : **2026-07-04 21:10:13 UTC, 16 secondes après la première occurrence** de l'erreur `STOCKFLOW-PWA-2` (échec d'hydratation React).
- La boucle complète a fonctionné sur un cas réel : cette erreur Sentry est la même anomalie que l'issue GitHub #32 du lot mobile (le diff d'hydratation capturé montre exactement le symptôme qualifié). Corrigée le 2026-07-13, zéro occurrence après déploiement, issue Sentry résolue avec un commentaire de traçabilité vers #32.
- Une erreur de vérification volontaire (`STOCKFLOW-PWA-1` , bouton de test dans l'UI) prouve le câblage de la chaîne de capture en production.

Les limites sont écrites dans la pièce 22 (init non conditionnée à l'environnement — le bruit de dev remonte tagué `production` ; capture client uniquement) et alimentent les recommandations du § 6.

3. Collecte et consignation des anomalies (C4.2.1)

Pièce détaillée : `docs/certification/14-plan-correction-bogues.md`.

Le processus, appliqué sans exception depuis l'issue #3 (2026-07-02) :

1. **Toute anomalie découverte devient une issue GitHub qualifiée avant correction** : constat, reproduction, cause (si connue), impact, sévérité. Pas de correctif « au passage » non tracé.
2. **Trois phases** : *Find* (détection — revue adversariale multi-agents avec réfutation indépendante, audits RGAA/Biome, retours terrain, alertes Sentry), *Verify* (qualification, dédoublonnage), *Triage* (catégorie Sécurité / Accessibilité / Fonctionnel / Dette, sévérité Critical → Low, planification).
3. **Suivi centralisé** sur le Kanban GitHub du projet (« StockFlow — Suivi qualité & certification »), colonnes À faire / En cours / Fait.
4. **Clôture commentée** : une issue est fermée avec le résumé du correctif réellement appliqué et sa vérification — jamais silencieusement.

Les sources d'entrée sont volontairement multiples, car chacune a un angle mort : la revue adversariale a trouvé les défauts de sécurité (7 findings du lot auth/RLS), la supervision Sentry a capturé l'erreur d'hydratation, et **le retour terrain** (test sur téléphone réel, 2026-07-13) a révélé ce que 36 tests e2e verts ne voyaient pas : le parcours mobile cassé au-delà de l'accueil (7 anomalies, issues #27-#33, corrigées le jour même). Trente-deux issues qualifiées au total à ce jour.

4. Fiche de consignation d'une anomalie réelle

Anomalie retenue : **#26**, la plus démonstrative parce qu'elle traverse tout le processus (détection tardive → qualification complète → correctif → mesure anti-récidive) et qu'elle porte sur l'outillage de maintenance lui-même.

Champ	Contenu
Identifiant	Issue GitHub #26
Titre	CI rouge depuis le 2026-07-03 : step Test échoue à l'import (APP_POSTGRES_URL manquant)
Date de détection	2026-07-12 (session housekeeping) — l'anomalie courait depuis le 2026-07-03
Détecteur	Revue interne des runs GitHub Actions (pas d'alerte : c'est le problème)
Symptôme	Tous les runs CI échouent depuis le lot RLS ; dernier run vert le 2026-07-02
Reproduction	Tout push sur main → job ci , step Test : Error: APP_POSTGRES_URL manquant : le runtime ne doit jamais se connecter avec POSTGRES_URL/DATABASE_URL (rôle postgres, BYPASSRLS)... — src/db/client.ts:16
Cause racine	Le client DB jette à l'import si APP_POSTGRES_URL est absent (garde fail-closed délibérée, à conserver). Les tests purs importent des modules qui importent ce client → toute la suite meurt au chargement. En local, vitest.config.ts charge .env.local : angle mort local/CI.
Impact	Aucune protection CI réelle pendant 9 jours (lint/typecheck/test/build jamais passés au vert) ; le statut rouge existait mais n'était pas visible dans le flux de travail
Sévérité	Haute (perte de garde-fou), sans impact production direct
Statut	Corrigée et fermée le 2026-07-12, avec mesure anti-récidive

5. Traitement d'une anomalie détectée

Le traitement de #26, étape par étape :

1. **Détection et qualification** (2026-07-12) : la fiche ci-dessus est rédigée dans l'issue **avant** toute correction — y compris la contrainte de conception : la garde fail-closed vient de la revue de sécurité (finding F8) et ne doit pas être affaiblie pour faire plaisir à la CI.
2. **Correctif** — dans le workflow uniquement, zéro code applicatif modifié : le step Test reçoit une `APP_POSTGRES_URL` factice (créer un pool `pg` ne se connecte pas tant qu'aucune requête ne part), et les tests d'intégration (`**/*.integration.test.ts`) sont exclus de la CI — ils exigent la base réelle, partagée avec la production, qui n'a pas sa place en CI (même politique que la suite e2e). Ils restent exécutés en local à chaque session.
3. **Vérification et livraison par le déploiement continu** : le correctif est un commit `fix:` poussé sur `main`, qui suit la chaîne standard du projet — CI GitHub Actions verte, puis déploiement Vercel automatique déclenché par le push, sans canal de livraison parallèle. Le run CI vert est constaté avant fermeture.
4. **Mesure anti-récidive** : un badge de statut CI est ajouté au README — la leçon de l'incident n'est pas « la CI a cassé » mais « **un rouge doit se voir** ». Depuis l'issue #34 (déploiements Vercel en échec, CI GitHub verte), la vérification du statut Vercel après push complète la même logique : chaque signal d'échec doit avoir un endroit où il est visible sans aller le chercher.
5. **Clôture commentée** : l'issue est fermée avec le résumé du correctif vérifié, alimentant le plan de correction des bogues (pièce 14).

6. Recommandations d'amélioration

Recentrées maintenance et supervision, par ordre de valeur, avec pour chacune un ordre de grandeur du coût de mise en œuvre (le coût monétaire est nul dans tous les cas — plans gratuits des services déjà utilisés) :

1. **Base de test séparée** (branche Supabase ou second projet) — la dette la plus sérieuse du projet : elle interdit aujourd'hui les tests d'intégration et e2e en CI. La lever ferait passer la CI de « tests purs » à « protection complète ». *Délai estimé : 1 à 2 jours (provisionnement, migrations, adaptation du sweep e2e et de la CI). Gain : détection des régressions d'accès aux données à chaque push, au lieu d'une exécution locale à la demande.*
2. **Conditionner l'init Sentry à l'environnement** et poser explicitement le tag `environment` — découvert en vérifiant la supervision (§ 2) : le bruit des sessions de dev remonte aujourd'hui tagué `production`, ce qui dégrade le signal des alertes. *Délai : moins d'une heure (deux lignes dans `__root.tsx`). Gain : alertes fiables, zéro faux volume.*
3. **Seuil de couverture en gate CI** sur la logique métier pure (le critère $\geq 80\%$ du Bloc 2, aujourd'hui vérifié à la main à chaque remesure). *Délai : une demi-journée (config `Vitest coverage.thresholds` ciblée sur `src/lib`). Gain : le critère devient auto-vérifié, une régression de couverture bloque le merge.*
4. **Recette e2e alignée sur les usages réels, rejouée périodiquement** — pas seulement à la livraison des features : la leçon du lot mobile (36 tests verts, parcours réel cassé) est qu'un cahier de recettes vieillit dès que l'usage évolue. *Délai : récurrent, ~1 h par mois. Gain : les angles morts du harnais sont découverts par la recette, pas par l'utilisateur.*

Une cinquième recommandation identifiée lors de la rédaction — supervision de disponibilité (ping de la production) — a été **mise en œuvre avant le dépôt** de ce dossier (§ 2, sonde uptime du 2026-07-15, délai réel : une heure) : la liste ci-dessus ne contient que ce qui reste à faire.

7. Journal des versions (C4.3.2)

Pièce source : `docs/certification/08-historique-versions.md` ; dernière version stable détaillée dans la pièce 20.

Version	Date	Tag git	Nouveautés	Anomalies corrigées
v0.2.0	2026-07-03	v0.2.0 (annoté)	CRUD équipements, scan, génération QR — sans auth ni RLS	Dette lint initiale #3 (13 erreurs → 0)
v0.3.0	2026-07-04	v0.3.0 (annoté)	Authentification JWT RS256 + RBAC + Row Level Security	7 correctifs de la revue de sécurité adversariale (commits <code>fix:</code> distincts et datés)
v0.4.0	2026-07-13	v0.4.0 (annoté)	Incidents, sécurité consolidée, accessibilité auditée, PWA offline. Version en production.	AN-1/AN-2 de la recette (#22, #23), 3 violations d'accessibilité (#25), sélecteur radio natif (#11), réparation CI (#26)

Le déploiement étant continu (push sur `main` = production), les correctifs entre deux jalons partent en production sans attendre le tag suivant — depuis `v0.4.0` : parcours mobile #27-#33 (dont l'erreur d'hydratation #32, § 2), build PWA sur Vercel #34. Chaque correctif déployé est documenté par son issue et son commit `fix:`.

Règles de tenue du journal :

- **Conventional Commits** (type anglais, description française), imposés par `commitlint` en hook `commit-msg` : l'historique est lisible et parsable, chaque tag annoté résume son lot.
- **Jamais de réécriture d'historique** pour masquer des corrections : les 7 correctifs post-revue de sécurité sont des commits `fix:` distincts et datés, pas un `rebase` cosmétique. Le journal documente le cycle qualité réel, pas un code parfait au premier jet.
- Un jalon significatif = un tag annoté `vX.Y.Z` poussé ; le retour arrière se fait par « Promote to Production » sur le déploiement Vercel précédent (instantané, sans toucher au journal).

8. Problème résolu avec le support client

Pièce détaillée : `docs/certification/23-support-client.md`.

Problème qualifié : le runtime se connecte via le pooler Supabase en **mode transaction** (Supavisor, port 6543), et toute la sécurité RLS repose sur des claims posés en `SET LOCAL` dans une transaction explicite. Deux garanties sont critiques et vérifiées empiriquement (15 tests d'intégration RLS) mais non confirmées officiellement : l'isolation des GUC `SET LOCAL` entre clients partageant le pool, et la remise à zéro de la connexion entre deux attributions. Pour un mécanisme de sécurité, s'appuyer sur un comportement observé plutôt que documenté est une dette.

Échange : question posée le 2026-07-15 sur le canal de support du plan gratuit (GitHub Discussions `supabase/supabase`, catégorie Questions) : <https://github.com/orgs/supabase/discussions/47946> — contexte complet, pattern `BEGIN; set_config(..., true); ...; COMMIT;`, politiques fail-closed, et la question de l'alternative en mode session.

Résolution : réponse reçue le 2026-07-15, moins de 12 h après l'ouverture. Le point clé dépasse la question posée : la garantie ne dépend pas du pooler mais du serveur Postgres lui-même — `set_config(..., true)` est annulé au commit/rollback par Postgres (documentation `SET` : la valeur ne vaut « que pour la transaction courante »), donc le GUC a déjà disparu quand la connexion peut être réattribuée. Le danger documenté du mode transaction concerne le `SET` de niveau *session*, que StockFlow n'utilise jamais. Le pattern est en outre celui de la stack Supabase elle-même (PostgREST injecte `request.jwt.claims` par le même mécanisme). Les deux points de vigilance retournés (toute requête dans la transaction porteuse de claims ; `current_setting(..., true)` en `missing_ok`) étaient déjà couverts par `withAuthContext` et les politiques fail-closed.

Décision : pattern confirmé et conservé tel quel, aucune modification de code ni d'infrastructure ; la dette « comportement observé, non documenté » est levée.

Contribution des parties prenantes : le candidat a qualifié le problème (lecture du code et de la configuration réelle, identification des deux garanties critiques) et posé la question argumentée ; le support communautaire Supabase (canal officiel du plan gratuit) a apporté la réponse technique et deux points de vigilance opérationnels ; la documentation PostgreSQL officielle sert de source normative (comportement de `SET LOCAL`) ; le remerciement a été posté et la réponse marquée comme acceptée — l'échange complet est public et consultable à l'URL du ticket.

Conclusion

La maintenance de StockFlow ne repose pas sur des intentions mais sur des mécanismes en place et éprouvés par de vrais incidents : des versions verrouillées et auditées, une supervision qui a réellement alerté en 16 secondes, un processus de consignation qui a absorbé 32 anomalies sans en perdre une, un journal de versions qui n'a jamais été réécrit, et un recours au support éditeur documenté. Les deux incidents racontés ici (#26, #32) ont chacun laissé une mesure anti-récidive derrière eux — c'est le critère d'une maintenance qui apprend, plutôt qu'une maintenance qui répare.

Annexe — Correspondance pièces officielles ↔ dossier technique

Pièce exigée (Bloc 4)	Ce dossier	Source dans le dépôt
Processus de mise à jour des dépendances	§ 1	07-referentiel-composants.md , 16-manuel-mise-a-jour.md
Système de supervision (C4.1.2)	§ 2	22-supervision-alerte.md , 09-securisation.md
Collecte/consignation des anomalies (C4.2.1)	§ 3	14-plan-correction-bogues.md
Fiche de consignation d'une anomalie	§ 4	Issue GitHub #26
Traitement d'une anomalie détectée	§ 5	Issue #26, rapport Bloc 2 § 1.2
Recommandations d'amélioration	§ 6	Rapport Bloc 2 (conclusion), pièce 22
Journal des versions (C4.3.2)	§ 7	08-historique-versions.md , 20-derniere-version-stable.md
Problème résolu avec le support client	§ 8	23-support-client.md